A MINIMAL FIELD HANDBOOK

The Minimal Classical ML Handbook

Linear models, trees, ensembles, kernels and metrics - the theory interviews test, in scikit-learn.

Mosafer

scikit-learn · XGBoost · leakage discipline

V1.0 - COMPACT EDITION

NOVEMBER 2026

Contents

1 Orientation **3**
- 1.1 The map . . . 3
- 1.2 The toolchain . . . 3
- 1.3 The running example . . . 3

2 Linear Models **5**
- 2.1 Linear regression . . . 5
- 2.2 Logistic regression: a classifier, not a regression . . . 5
- 2.3 Regularization: ridge, LASSO, elastic net . . . 5
- 2.4 The lens worth internalizing . . . 6

3 Bias–Variance and Validation **7**
- 3.1 The decomposition . . . 7
- 3.2 Cross-validation . . . 7
- 3.3 Learning curves . . . 8

4 Trees and Ensembles **9**
- 4.1 Decision trees . . . 9
- 4.2 Random forests: bagging . . . 9
- 4.3 Gradient boosting: the residual learner . . . 9

5 SVMs and Kernels **11**
- 5.1 The margin objective . . . 11
- 5.2 The kernel trick . . . 11
- 5.3 Where SVMs still win . . . 11

6 Unsupervised Learning **13**
- 6.1 k-means . . . 13
- 6.2 PCA . . . 13
- 6.3 Embedding plots: t-SNE and UMAP . . . 14

7 Metrics **15**
- 7.1 Classification metrics . . . 15
- 7.2 Thresholds are business decisions . . . 15
- 7.3 Regression metrics . . . 16
- 7.4 Imbalance: fix the measurement, then the training . . . 16

8 scikit-learn Discipline **17**
- 8.1 The Pipeline . . . 17
- 8.2 Tuning without leaking . . . 17
- 8.3 Leakage: the taxonomy . . . 18

A Cheat Sheet **19**
- A.1 Linear models . . . 19
- A.2 Validation . . . 19
- A.3 Trees and boosting . . . 19
- A.4 Metrics . . . 19
- A.5 Unsupervised . . . 20

A.6 Leakage checklist 20

CHAPTER 1

Orientation

Classical machine learning — linear models, trees, ensembles, kernels — is where interviews live and where tabular problems are solved. Neural nets win on perceptual data; on the tabular benchmarks that fill business datasets, gradient-boosted trees still routinely win. This handbook is the working set, with the theory interviewers probe.

1.1 The map

Family	Members in this book
linear models	linear / logistic / softmax regression, ridge, LASSO (Ch. 2)
the learning-theory lens	bias–variance, cross-validation, learning curves (Ch. 3)
trees and ensembles	decision trees, random forests, gradient boosting / XGBoost (Ch. 4)
margin methods	SVMs and the kernel trick (Ch. 5)
unsupervised	k-means, PCA, embedding caveats (Ch. 6)
measurement	precision/recall, ROC, calibration, imbalance (Ch. 7)
discipline	`sklearn` pipelines and leakage prevention (Ch. 8)

1.2 The toolchain

```
pip install scikit-learn pandas numpy matplotlib xgboost
```

`scikit-learn`'s API is the standard grammar — `fit` / `predict` / `transform` / `Pipeline` — and this book assumes it throughout.

1.3 The running example

```
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split

X, y = fetch_openml("adult", version=2, as_frame=True, return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=0)
print(X_tr.shape, y_tr.value_counts(normalize=True).round(3).to_dict())
```

The Adult income dataset: 48k rows, mixed numeric and categorical features, a binary target with class imbalance — every chapter's technique applies to it, and every chapter's mistake is a mistake someone made on it.

> **PITFALL**
> Classical ML's failure mode is silent data leakage: a feature that encodes the answer (a timestamp of *when the label was recorded*), scaling fit on the test set, a random split of time-ordered data. The models

still train; the metrics still look great — until deployment. Ch. 8 is the discipline that prevents this, and Ch. 3 is the honesty that detects it.

WHY THIS MATTERS FOR AI POSITIONS

Interviewers use classical ML to check whether a candidate understands *learning* or only *frameworks*: bias–variance, why regularization works, what AUC hides, when a tree beats a network. The PyTorch handbook proves you can train; this one proves you know what training means.

CHAPTER 2

Linear Models

Linear models fit a hyperplane; their power is in the lens: losses, regularizers, and the geometry of what each penalty does. They train in milliseconds, interpret cleanly, and are the strongest baseline you must beat before reaching for anything deeper.

2.1 Linear regression

$$\hat{y} = w^\top x + b, \qquad \mathcal{L}(w, b) = \frac{1}{n}\sum_i (w^\top x_i + b - y_i)^2$$

MSE is convex; the closed form exists ($w = (X^\top X)^{-1} X^\top y$) but `sklearn` solves it stably:

```
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

model = make_pipeline(StandardScaler(), LinearRegression())
model.fit(X_tr[numeric_cols], y_tr_num)
print(model[-1].coef_)          # standardized: coefficients are comparable
```

Standardize first: with mixed-scale features, raw coefficients compare apples to radios.

2.2 Logistic regression: a classifier, not a regression

$$\Pr(y = 1 \mid x) = \sigma(w^\top x + b), \qquad \mathcal{L} = -\frac{1}{n}\sum_i \left[y_i \log p_i + (1 - y_i) \log(1 - p_i) \right]$$

Cross-entropy loss, convex, gradient $p - y$ (the Optimization handbook's cleanest gradient). Multiclass extends to softmax. The output is a *probability* — which makes calibration (Ch. 7) part of the modeling.

2.3 Regularization: ridge, LASSO, elastic net

Penalty	Term	Effect
ridge (L_2)	$+\lambda \sum w_j^2$	shrinks all weights; handles correlated features by sharing
LASSO (L_1)	$+\lambda \sum \lvert w_j \rvert$	drives weights to exact zero — built-in feature selection
elastic net	both	L_1 sparsity with L_2 stability on correlated groups

```
from sklearn.linear_model import RidgeCV, LassoCV

ridge = make_pipeline(StandardScaler(), RidgeCV(alphas=np.logspace(-3, 3, 13)))
ridge.fit(X_tr[numeric_cols], y_tr_num)
print(ridge[-1].alpha_)                    # chosen by internal CV

lasso = make_pipeline(StandardScaler(), LassoCV(cv=5))
```

```
lasso.fit(X_tr[numeric_cols], y_tr_num)
kept = X_tr[numeric_cols].columns[lasso[-1].coef_ != 0]
print(kept.tolist())                        # LASSO selected features
```

λ is tuned by cross-validation on a log grid — never by hand. The geometry: L_2's round constraints shrink smoothly; L_1's diamond corners sit on axes, so optima land at exact zeros (Optimization handbook, Ch. 7).

2.4 The lens worth internalizing

- **Bias–variance**: λ moves you along the trade-off — more penalty, more bias, less variance (Ch. 3 makes it measurable).
- **Linear model + good features** is the strongest interpretability tool in ML: coefficients are marginal effects *under the standardization*, with caveats under multicollinearity.
- **Logistic regression is a calibrated probability model by construction** — until you maul it with class weights; then recalibrate (Ch. 7).

> **PITFALL**
>
> Three linear-model classics: (1) one-hot encoding high-cardinality categoricals into hundreds of collinear columns — use target or frequency encoding, or regularization, and say which in the README; (2) interpreting coefficients causally — "more education adds $X" is only true under assumptions linear regression does not check; (3) fitting the scaler on the full dataset before splitting — test statistics leaked into training (Ch. 8's first commandment).

CHAPTER 3

Bias–Variance and Validation

Two questions govern every modeling decision: *how wrong can this model be?* (bias + variance) and *how do I know?* (validation). This chapter makes both measurable.

3.1 The decomposition

Expected test error of a model class, over training sets:

$$\text{Error} = \underbrace{\text{Bias}^2}_{\text{wrong assumptions}} + \underbrace{\text{Variance}}_{\text{sensitivity to the sample}} + \underbrace{\sigma^2}_{\text{irreducible noise}}$$

Model capacity moves you along the curve: more capacity lowers bias and raises variance. The U-shaped test error vs. capacity plot is the single most reused figure in ML — regularization, early stopping, pruning, and ensembling are all levers on its axes.

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import validation_curve

degrees = range(1, 15)
tr_scores, va_scores = validation_curve(
    make_pipeline(PolynomialFeatures(), LinearRegression()),
    x[:, None], y, param_name="polynomialfeatures__degree",
    param_range=degrees, cv=5, scoring="r2")
# plot tr_scores.mean(1) and va_scores.mean(1): the U-shape, live
```

3.2 Cross-validation

k-fold CV estimates out-of-sample performance by rotating the holdout fold. Variants that matter more than people admit:

Variant	When
`KFold(shuffle=True, seed)`	default, i.i.d. data
`StratifiedKFold`	classification — preserve class ratios per fold
`TimeSeriesSplit`	temporal data — train strictly before test, never shuffle
`GroupKFold`	repeated entities (many rows per patient/user) — one entity never spans train and test
`RepeatedStratifiedKFold`	small datasets — average out the fold lottery

```
from sklearn.model_selection import cross_validate, StratifiedKFold

cv = StratifiedKFold(5, shuffle=True, random_state=0)
res = cross_validate(model, X_tr, y_tr, cv=cv,
                     scoring=["roc_auc", "average_precision"],
```

```
                    return_train_score=True)
print(res["test_roc_auc"].mean().round(3),
      "+-", res["test_roc_auc"].std().round(3))
print("train-test gap:", (res["train_roc_auc"].mean()
                          - res["test_roc_auc"].mean()).round(3))
```

The *train-test gap* inside CV is the variance diagnostic: a wide gap is overfitting; both scores low is bias (or bad features).

3.3 Learning curves

```
from sklearn.model_selection import learning_curve

sizes, tr, va = learning_curve(model, X_tr, y_tr, cv=cv,
                               train_sizes=np.linspace(0.1, 1.0, 8))
```

- train and validation curves *converging high*: bias — need a richer model or features, not more data.
- converging low: you are done.
- gap *closing* with more data: variance — more data (or regularization) will pay.

PITFALL

Nested leakage in tuning: selecting hyperparameters on the same CV folds you report, or reusing the test set across iterations, turns CV into training. The firewall: *test set untouched until the end* (Statistics handbook, Ch. 4's one-touch rule), hyperparameters chosen by CV inside the training set only, and the final number from one test evaluation.

THE PRACTICAL DEFAULT

Stratified 5-fold with a fixed seed for everything small; TimeSeriesSplit or GroupKFold when structure demands it; learning curves before architecture changes; the test set spent exactly once. This paragraph prevents most classical-ML paper retractions.

CHAPTER 4

Trees and Ensembles

Gradient-boosted trees are the state of the art on tabular data — faster to train than neural nets, stronger out of the box, and the reason most "we replaced XGBoost with deep learning" projects quietly switch back. This chapter builds trees, then the two ensemble ideas.

4.1 Decision trees

Recursive partitioning: split on the feature/threshold that best purifies the children (Gini impurity or entropy for classification, variance reduction for regression), recurse, stop at depth/leaf-size.

```
from sklearn.tree import DecisionTreeClassifier, export_text

tree = DecisionTreeClassifier(max_depth=3, min_samples_leaf=50,
                              random_state=0)
tree.fit(X_tr_num, y_tr)
print(export_text(tree, feature_names=list(X_tr_num.columns)))
```

A single deep tree overfits perfectly (zero training error, arbitrary variance) — `max_depth` and `min_samples_leaf` are the regularization. The printed rules are the interpretability feature: audit them, and you audit the model.

4.2 Random forests: bagging

Bootstrap many trees on random feature subsets; average. Variance drops roughly with independence — that is the whole theory:

$$\mathrm{Var}(\text{forest}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$$

where ρ is correlation between trees — `max_features` is the dial that decorrelates them (lower ρ, at some bias cost). Forests are hard to overfit (more trees never hurt, only cost) and give free out-of-bag estimates:

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(n_estimators=500, max_features="sqrt",
                            min_samples_leaf=2, n_jobs=-1,
                            oob_score=True, random_state=0)
rf.fit(X_tr_num, y_tr)
print("OOB:", rf.oob_score_.round(3))      # free validation estimate
```

4.3 Gradient boosting: the residual learner

Boosting builds trees *sequentially*, each fitting the residuals of the ensemble so far — gradient descent in function space:

$$F_m(x) = F_{m-1}(x) + \nu \cdot h_m(x), \qquad h_m \approx -\nabla_F \mathcal{L}(F_{m-1})$$

with learning rate ν (shrinkage) and h_m a small tree. The modern implementations add second-order information, histogram-based splits, and regularization:

```python
from xgboost import XGBClassifier

xgb = XGBClassifier(n_estimators=400, max_depth=6, learning_rate=0.05,
                    subsample=0.8, colsample_bytree=0.8,
                    reg_lambda=1.0, tree_method="hist",
                    eval_metric="logloss", early_stopping_rounds=50,
                    random_state=0)
xgb.fit(X_tr_num, y_tr_num, eval_set=[(X_te_num, y_te_num)], verbose=50)
```

Knob	Effect
`learning_rate` with `n_estimators`	the coupled pair: lower rate needs more trees — tune together
`max_depth` (3–8)	per-tree capacity; 4–6 for most tabular problems
`subsample`, `colsample_bytree`	stochastic regularization — decorrelates trees
`reg_lambda / reg_alpha`	L_2 / L_1 on leaf weights
`early_stopping_rounds`	stop when the eval metric stops improving — free model selection

PITFALL

Boosting *can* overfit, unlike bagging — early stopping on a validation set is mandatory, not optional. And both tree methods take raw, unscaled, missing-valued numeric inputs happily: scaling is needless, and imputation is often handled internally — pipelines built for linear models do not transfer unchanged (Ch. 8).

FEATURE IMPORTANCE, HANDLED WITH CARE

`feature_importances_` is bias-prone (high-cardinality features win); prefer permutation importance on a held-out fold — and remember importance is *descriptive*, not causal. SHAP values are the modern default when stakeholders want per-prediction explanations.

CHAPTER 5

SVMs and Kernels

Support vector machines frame classification geometrically: find the separating hyperplane with maximum margin. They contribute two ideas that outlive their usage — margins, and the kernel trick.

5.1 The margin objective

$$\min_{w,b} \tfrac{1}{2}\|w\|^2 + C\sum_i \xi_i \quad \text{s.t.} \quad y_i(w^\top x_i + b) \geq 1 - \xi_i$$

Maximize the margin ($1/\|w\|$) while penalizing margin violations ξ_i with strength C. The dual solution has $w = \sum_i \alpha_i y_i x_i$ with $\alpha_i > 0$ only for the *support vectors* — points on or inside the margin; every other training point could be deleted without changing the model.

```
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

svm = make_pipeline(StandardScaler(),
                    SVC(kernel="rbf", C=1.0, gamma="scale"))
svm.fit(X_tr_num[:5000], y_tr[:5000])       # O(n^2)-O(n^3): sample on big data
```

5.2 The kernel trick

The dual touches the data only through inner products $\langle x_i, x_j \rangle$. Replace them with a kernel $K(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle$ and the linear model acts in the feature space ϕ without ever computing it:

Kernel	Geometry
linear	hyperplane in the original space
polynomial	interactions of degree d
RBF (Gaussian)	infinite-dimensional: local similarity — `gamma` sets the neighborhood scale

`C` and `gamma` form the classic 2-D search: small `gamma` = smooth, large = wiggly; small `C` = wide margin with more violations, large = hard margin. `GridSearchCV` over a log-log grid (Ch. 3's CV) is the recipe.

5.3 Where SVMs still win

- small-to-medium data (n up to tens of thousands) with strong feature engineering;
- text classification with TF-IDF — linear SVMs remain embarrassingly strong baselines;
- anytime the margin interpretation helps stakeholder communication.

Beyond that, kernel methods ceded ground to gradient boosting (tabular) and neural networks (perception) — mostly for scale and representation-learning reasons, not because the margin idea failed.

> **PITFALL**

SVMs need scaled features (distances drive the kernel), and their training cost is at least quadratic in n — subsample for prototyping. `probability=True` enables Platt scaling for probabilities but multiplies cost via internal CV; if you need probabilities at scale, prefer logistic regression or calibrated boosting (Ch. 7).

THE IDEA THAT TRANSFERRED

Kernels = "learn with inner products, choose the geometry later". The same abstraction powers Gaussian processes (a kernel plus Bayesian regression), and attention in Transformers is a data-dependent kernel over tokens — $\text{softmax}(qk^\top)v$ *is* a kernel smoother. The SVM chapter is where that abstraction enters the canon.

CHAPTER 6

Unsupervised Learning

No labels, so no ground truth to grade against — unsupervised learning is judged by usefulness: do the clusters mean something, does the projection preserve the structure you care about?

6.1 k-means

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

Xs = StandardScaler().fit_transform(X_num)
km = KMeans(n_clusters=4, n_init=10, random_state=0).fit(Xs)
print(km.inertia_, np.bincount(km.labels_))
```

Objective: minimize within-cluster squared distance. Mechanics: init centroids (`n_init` restarts fight bad initializations — k-means++ is the smart default), assign, update, repeat until stable.

- **Choosing k**: elbow on inertia (visual), silhouette score (geometric), or downstream usefulness (the honest criterion — clusters exist to be used).
- **Known limits**: assumes spherical, equal-variance clusters; the mean is meaningful in your space (one-hot heavy data breaks it). `DBSCAN` for arbitrary shapes and outlier detection; `GaussianMixture` for soft, elliptical clusters.

6.2 PCA

Principal component analysis projects data onto orthogonal directions of maximal variance — equivalently (and more usefully), the SVD of the centered data matrix:

$$X = U\Sigma V^\top, \qquad \text{PCs} = XV_k$$

```
from sklearn.decomposition import PCA

pca = PCA(n_components=0.95, random_state=0)   # keep 95% variance
X_pca = pca.fit_transform(StandardScaler().fit_transform(X_num))
print(pca.explained_variance_ratio_.round(3),
      "components:", pca.n_components_)
```

- **compression / denoising**: keep components explaining 95–99% variance; reconstruct to drop noise dimensions.
- **preprocessing**: decorrelate features before linear models; speed up k-means.
- **interpretation**: loadings (`pca.components_`) name what each PC measures — one glance at the loadings matrix beats staring at scatter plots.

6.3 Embedding plots: t-SNE and UMAP

```
from sklearn.manifold import TSNE
import umap

Z_tsne = TSNE(n_components=2, perplexity=30,
              random_state=0).fit_transform(X_pca)
Z_umap = umap.UMAP(n_neighbors=15, min_dist=0.1,
                   random_state=0).fit_transform(X_pca)
```

Both build beautiful 2-D pictures with *non-uniform* fidelity: local neighborhoods are roughly preserved, global distances and cluster sizes are not. Consequences people forget: inter-cluster distance in a t-SNE plot is not meaningful, cluster sizes are not population sizes, and perplexity changes the picture qualitatively.

PITFALL

Three unsupervised classics: (1) running k-means on unscaled data — the feature with the biggest units picks all the centroids; (2) interpreting t-SNE clusters as natural kinds — clusters can be artifacts of perplexity; validate against something external; (3) fitting PCA/scalers on the full dataset before splitting — the unsupervised version of leakage, and it happens constantly (Ch. 8's rule: fit transforms on train folds only).

CONNECTION TO THE MODERN STACK

PCA is the SVD of the covariance structure (the Math handbook, Ch. 2); embeddings from Ch. 2 of the RAG handbook are learned non-linear PCA-like maps; k-means over LLM embeddings is the cheapest clustering stack in existence. The classical tools did not die — they moved one layer down.

CHAPTER 7

Metrics

The metric *is* the objective you are optimizing for the business. Choosing it is a modeling decision; here is the map, with the traps each metric hides.

7.1 Classification metrics

Metric	Question it answers
accuracy	overall — *misleads under imbalance* (99% accuracy with 1% positives = predict "no")
precision	of predicted positives, how many are right? (cost of a false alarm)
recall / TPR	of actual positives, how many caught? (cost of a miss)
F_1	harmonic mean — balances precision and recall
ROC-AUC	ranking quality across all thresholds; can look noble on rare classes
PR-AUC	precision-recall area — **the** metric for imbalanced problems

```
from sklearn.metrics import (precision_recall_curve, roc_auc_score,
                             average_precision_score, classification_report)

proba = model.predict_proba(X_te)[:, 1]
print("ROC-AUC:", roc_auc_score(y_te, proba).round(3),
      "PR-AUC:", average_precision_score(y_te, proba).round(3))
prec, rec, thr = precision_recall_curve(y_te, proba)
```

7.2 Thresholds are business decisions

The model outputs probabilities; the *operating threshold* converts them into actions — and it belongs to the product, not the model. Pick it from the precision-recall curve against real costs:

```
target_recall = 0.90
i = np.argmin(np.abs(rec[:-1] - target_recall))
print(f"threshold {thr[i]:.3f} -> precision {prec[i]:.3f} at recall {rec[i]:.3f}")
```

Then verify calibration — does 0.8 mean 80%? — because thresholds on uncalibrated scores are arithmetic on fiction:

```
from sklearn.calibration import CalibratedClassifierCV
calibrated = CalibratedClassifierCV(model, method="isotonic", cv=5)
```

7.3 Regression metrics

Metric	Sensitivity
MAE	linear in errors — robust to outliers
RMSE	squared — punishes large errors (use when large errors are the point)
R^2	variance explained — no absolute meaning across problems
MAPE	relative — explodes near zero (never for values near 0)

7.4 Imbalance: fix the measurement, then the training

- Report PR-AUC and per-class recall — accuracy first becomes meaningless at 100:1.
- `class_weight="balanced"` or resampling (SMOTE for synthesis) — they trade precision for recall at the loss level.
- Threshold-tune on validation per class, never on test.
- Beware metrics that reward the majority: a confusion matrix printed at the chosen threshold is the honest artifact.

PITFALL

The metric traps that end interviews early: (1) ROC-AUC on a 0.1% positive-rate problem looks stellar while PR-AUC tells the truth — always plot the PR curve for rare positives; (2) reporting accuracy *after* class weighting — the weighting moved the operating point, so the accuracy is not comparable to the unweighted baseline; (3) optimizing a proxy (log-loss) while reporting a different metric (AUC) without checking they moved together — they usually do, until calibration breaks and they do not.

THE ONE HABIT

Write the metric definition — including the threshold and the class weights — into the experiment config next to the seed. Six months later, "we improved F1" without those three numbers is not a claim; it is a memory of a claim.

CHAPTER 8

scikit-learn Discipline

Every chapter so far had one forbidden move in it: fitting a transform on data the model will be validated on. `sklearn`'s `Pipeline` makes the correct thing the default thing — this chapter is the discipline chapter.

8.1 The Pipeline

A pipeline chains transformers and an estimator; `fit` runs the chain, and *every transform's parameters are learned on the training folds only*:

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_validate, StratifiedKFold

num_pipe = Pipeline([("impute", SimpleImputer(strategy="median")),
                     ("scale", StandardScaler())])
cat_pipe = Pipeline([("impute", SimpleImputer(strategy="most_frequent")),
                     ("onehot", OneHotEncoder(handle_unknown="ignore"))])

pre = ColumnTransformer([("num", num_pipe, num_cols),
                         ("cat", cat_pipe, cat_cols)])
model = Pipeline([("pre", pre),
                  ("clf", LogisticRegression(max_iter=1000,
                                             class_weight="balanced"))])

cv = StratifiedKFold(5, shuffle=True, random_state=0)
res = cross_validate(model, X_tr, y_tr, cv=cv, scoring="roc_auc")
```

Cross-validating the *pipeline* — not a pre-transformed matrix — is the entire point: the scaler sees only the training folds of each split, imputation statistics too, and the one-hot encoder handles unseen categories gracefully at predict time.

8.2 Tuning without leaking

`GridSearchCV` / `RandomizedSearchCV` nest the tuning CV inside the outer evaluation CV — hyperparameter selection and performance estimation stay separated:

```
from sklearn.model_selection import GridSearchCV

grid = GridSearchCV(model, param_grid={
        "clf__C": [0.01, 0.1, 1.0, 10.0],
        "pre__num__impute__strategy": ["mean", "median"]},
    cv=5, scoring="roc_auc", n_jobs=-1)
grid.fit(X_tr, y_tr)                    # tuning, inside training data
print(grid.best_params_, grid.best_score_.round(3))
```

```
final = cross_validate(grid.best_estimator_, X_tr, y_tr,
                       cv=StratifiedKFold(5, shuffle=True,
                                          random_state=1),
                       scoring="roc_auc")  # honest estimate
```

The best score from `GridSearchCV` is *optimistic* (it selected among candidates); the outer-CV estimate of `grid.best_estimator_` is the number that goes in the paper.

8.3 Leakage: the taxonomy

Leak	Example
preprocessing leakage	scaler/imputer fit on train + test combined
feature leakage	any feature computed using information unavailable at prediction time (post-outcome timestamps, aggregates over the future)
split leakage	random split of grouped or temporal data
selection leakage	reporting the best of many test-set evaluations
target leakage via IDs	row order, patient IDs, near-duplicate rows across splits

PITFALL

The leakiest object in classical ML is the analyst with one extra idea at 5pm. Institutional defenses: features get a `date_available` column and every feature is justified against it; the test set lives in a separate file touched by one function; and every experiment logs its config (the Git handbook, Ch. 7) so the leak is at least traceable after the fact.

THE NOTEBOOK-TO-PRODUCTION BRIDGE

The `Pipeline` object *is* the deployment artifact: `joblib.dump(pipeline, "model.joblib")` serializes preprocessing and model in one object, and the serving path calls `pipeline.predict(raw_row)` — no train/serve skew, no "forgot the scaler" incidents. Build pipelines from day one; the production refactor you skip is the skew you ship.

APPENDIX A

Cheat Sheet

A.1 Linear models

Task	Code
Fit	`make_pipeline(StandardScaler(), LogisticRegression())`
Ridge / LASSO	`RidgeCV(alphas=logspace(-3, 3, 13))` / `LassoCV(cv=5)`
Selected features	`cols[lasso[-1].coef_ != 0]`
Balance	`class_weight="balanced"`

A.2 Validation

Task	Code
CV	`cross_validate(model, X, y, cv=StratifiedKFold(5, shuffle=True, random_state=0))`
Temporal	`TimeSeriesSplit(n_splits=5)` — never shuffle
Grouped	`GroupKFold(n_splits=5)`
Curves	`learning_curve`, `validation_curve`
Tune	`GridSearchCV(pipeline, grid, cv=5)` then outer-CV the winner

A.3 Trees and boosting

Task	Code
Forest	`RandomForestClassifier(n_estimators=500, max_features="sqrt", oob_score=True)`
Boost	`XGBClassifier(learning_rate=0.05, max_depth=6, subsample=0.8, early_stopping_rounds=50)`
Read a tree	`export_text(tree, feature_names=...)`
Importance	permutation importance on held-out data; SHAP for per-prediction

A.4 Metrics

Task	Code
Ranking	`roc_auc_score(y, proba)`, `average_precision_score(y, proba)`
Curve	`precision_recall_curve(y, proba) \to` pick threshold by cost
Calibrate	`CalibratedClassifierCV(model, method="isotonic")`
Report	`classification_report(y_true, y_pred)`

A.5 Unsupervised

Task	Code
Cluster	`KMeans(n_clusters=k, n_init=10, random_state=0)` on scaled data
Project	`PCA(n_components=0.95)` on scaled data; `explained_variance_ratio_`
Embed (2-D)	`TSNE(perplexity=30)` / `UMAP(...)` — local structure only

A.6 Leakage checklist

1. Every transform fit inside the pipeline, inside the fold.
2. Features justified against a `date_available` stamp.
3. Splits respect time and groups.
4. Test set touched exactly once, at the end.
5. Config + seed + metric definition logged with every result.

> **THE HABITS THAT MATTER**
>
> Pipeline everything; stratify everything; spend the test set once; prefer PR-AUC on imbalanced data; tune the threshold like the business decision it is; and write the metric definition into the config. Trees for tabular, linear for interpretability, leakage discipline for everything — that is classical ML.